

Лабораторная работа №12,
Программирование в командном
процессоре ОС UNIX. Командные
файлы

Архитектура компьютера и операционные системы

Цель работы

Изучить основы программирования в оболочке ОС UNIX/Linux. Получить практические навыки написания командных файлов Bash, передачи параметров командной строки, работы с файлами и каталогами, а также использования управляющих конструкций языка Bash.

Задание

Выполнить задания раздела **10.3 «Последовательность выполнения работы»**:

1. Написать скрипт, который при запуске делает резервную копию самого себя в каталог `backup` домашнего каталога пользователя и архивирует файл одним из архиваторов (`zip`, `bzip2` или `tar`).
 2. Написать пример командного файла, обрабатывающего произвольное число аргументов командной строки, в том числе превышающее десять.
 3. Написать командный файл — аналог команды `ls` (без использования команд `ls` и `dir`), выводящий информацию о каталоге и о возможностях доступа к файлам.
 4. Написать командный файл, который получает в качестве аргумента формат файла (`.txt`, `.doc`, `.jpg`, `.pdf` и т.д.) и вычисляет количество таких файлов в указанной директории.
-

Теоретическое введение

Командная оболочка **Bash** представляет собой интерпретатор команд, позволяющий выполнять команды операционной системы, использовать переменные, циклы, условия и писать командные файлы.

В ходе лабораторной работы используются следующие возможности Bash:

- позиционные параметры (\$1, \$2, \$#);
- команда `shift`;
- цикл `for`;
- команда `test`;
- метасимвол `*`;
- работа с файлами и каталогами.

В методических указаниях указано, что:

`echo *` выводит имена всех файлов текущего каталога и представляет собой простейший аналог команды `ls`.

Также в работе используются проверки:

- `test -d` — каталог;
 - `test -f` — обычный файл;
 - `test -r` — доступен по чтению;
 - `test -w` — доступен по записи.
-

Выполнение лабораторной работы

Подготовка среды

1. Открытие терминала

В среде **GitHub Codespaces** открыть встроенный терминал.

2. Проверка текущего каталога

`pwd`

```
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab12 (develop) $ pwd
/workspaces/study_2025-2026_os-intro/labs/lab12
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab12 (develop) $
```

3. Создание файла скрипта

`touch backup.sh`

```
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab12 (develop) $ ls
Report_lab12.qmd backup.sh
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab12 (develop) $
```

4. Открытие файла

Открыть созданный файл в редакторе и вставить исходный код.

```
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab12
(develop) $ mcedit backup.sh
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab12 (develop) $
```

5. Назначение права выполнения

`chmod +x backup.sh`

```
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab12 (develop) $ chmod +x backup.sh
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab12 (develop) $
```

6. Запуск скрипта

`./backup.sh`

```
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab12 (develop) $ ./backup.sh
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab12 (develop) $
```

Задание 1. Резервная копия самого себя

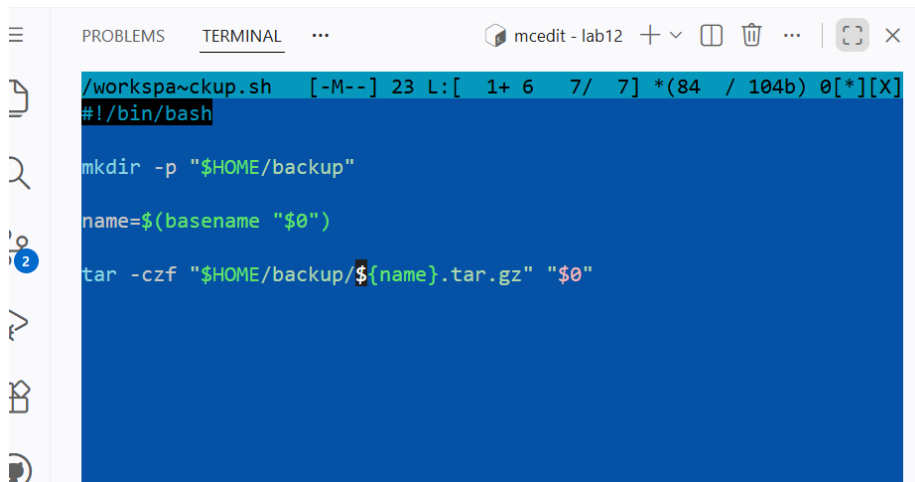
Исходный код

```
#!/bin/bash

mkdir -p "$HOME/backup"

name=$(basename "$0")

tar -czf "$HOME/backup/${name}.tar.gz" "$0"
```



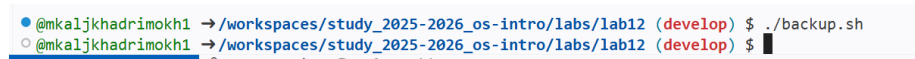
Пояснение

Скрипт:

- создаёт каталог backup в домашнем каталоге;
- получает собственное имя через \$0;
- архивирует самого себя в формате tar.gz.

Запуск

```
chmod +x backup.sh
./backup.sh
```



Проверка результата

```
cd ~/backup
echo *
```

```
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab12 (develop) $ cd ~/backup
@mkaljkhadrimokh1 → ~/backup $ echo *
backup.sh.tar.gz
@mkaljkhadrimokh1 → ~/backup $
```

```
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab12 (
develop) $ ./args.sh one two three four five six seven eight nine ten
eleven
two
three
four
five
six
seven
eight
nine
ten
eleven
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab12 (
develop) $
```

Задание 3. Аналог команды ls

Исходный код

```
#!/bin/bash

cd "$1" || exit 1

for f in *
do
    echo -n "$f : "

    if test -d "$f"
    then
        echo -n "directory "
    else
        echo -n "file "
    fi

    if test -r "$f"
    then
        echo -n "readable "
    fi

    if test -w "$f"
    then
        echo -n "writable "
    fi

    echo
done
```

```
@mkaljkhadrimokh1 →/workspaces/study_2025-2026_os-intro/labs/lab12 (
● develop) $ touch nols.sh
@mkaljkhadrimokh1 →/workspaces/study_2025-2026_os-intro/labs/lab12 (
@mkaljkhadrimokh1 →/workspaces/study_2025-2026_os-intro/labs/lab12 (
○ develop) $ mcedit nols.sh
```

```

/workspaces/study_2025-2026_os-intro/labs/lab12/nols.sh [-M--] 8 L:[ 1+25 26/ 27] *(304
#!/bin/bash

cd "$1" || exit 1

for f in *
do
    echo -n "$f : "

    if test -d "$f"
    then
        echo -n "directory "
    else
        echo -n "file "
    fi

    if test -r "$f"
    then
        echo -n "readable "
    fi

    if test -w "$f"
    then
        echo -n "writable "
    fi

    echo
done

```

Пояснение

Используется:

- `for f in *` — перебор всех файлов каталога;
- `test -d`, `test -r`, `test -w` — определение типа файла и прав доступа.

Пример запуска

```

chmod +x nols.sh
./nols.sh .

```

```

@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab12 (develop) $ chmod +x nols.sh

```

```

● tro/labs/lab12 (develop) $ ./nols.sh
Report_lab12.qmd : file readable writable
args.sh : file readable writable
backup.sh : file readable writable
nols.sh : file readable writable
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-intro/labs/lab12 (develop) $

```

Задание 4. Подсчёт файлов по расширению

Исходный код

```
#!/bin/bash
```

```
ext="$1"
```

```
dir="$2"
```

```
count=0
```

```
cd "$dir" || exit 1
```

```
for f in *"$ext"
```

```
do
```

```
    if test -f "$f"
```

```
    then
```

```
        count=$((count + 1))
```

```
    fi
```

```
done
```

```
echo "$count"
```

```
-----  
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-in  
tro/labs/lab12 (develop) $ touch count.sh
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS + v ... [ ]
/workspa~ount.sh [-M--] 4 L:[ 1+ 0 1/ 18][*][X]
#!/bin/bash

ext="$1"
dir="$2"

count=0

cd "$dir" || exit 1

for f in *"$ext"
do
    if test -f "$f"
    then
        count=$((count + 1))
    fi
done

echo "$count"
```

Пример запуска

```
chmod +x count.sh
./count.sh .txt .
```

```
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-in
● tro/labs/lab12 (develop) $ chmod +x count.sh
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-in
● tro/labs/lab12 (develop) $ ./count.sh .txt
0
@mkaljkhadrimokh1 → /workspaces/study_2025-2026_os-in
```

Выводы

В ходе выполнения лабораторной работы были изучены основные средства программирования в командной оболочке Bash.

Получены практические навыки:

- создания и запуска командных файлов;
 - передачи аргументов командной строки;
 - использования циклов и условий;
 - работы с файлами и каталогами;
 - проверки прав доступа к файлам.
-

Контрольные вопросы

1. Объясните понятие командной оболочки. Приведите примеры командных оболочек. Чем они отличаются?

Командная оболочка — это программа, обеспечивающая взаимодействие пользователя с операционной системой посредством ввода команд.

Примеры: sh, csh, ksh, bash.

Они отличаются синтаксисом, набором встроенных команд и возможностями программирования.

2. Что такое POSIX?

POSIX — это набор стандартов, обеспечивающих совместимость UNIX/Linux-подобных операционных систем и переносимость программ.

3. Как определяются переменные и массивы в языке программирования bash?

`name=value`

4. Каково назначение операторов let и read?

let выполняет арифметические вычисления.

read считывает значения со стандартного ввода.

5. Какие арифметические операции можно применять в языке программирования bash?

Сложение, вычитание, умножение, деление, остаток от деления, логические и побитовые операции.

6. Что означает операция (())?

Это форма записи арифметических выражений и условий в Bash.

7. Какие стандартные имена переменных Вам известны?

PATH, HOME, IFS, MAIL, TERM, LOGNAME, PS1, PS2.

8. Что такое метасимволы?

Метасимволы — специальные символы оболочки, имеющие служебный смысл.

Примеры: *, ?, [], |, <, >.

9. Как экранировать метасимволы?

С помощью обратной косой черты \, одинарных или двойных кавычек.

10. Как создавать и запускать командные файлы?

```
chmod +x file.sh
./file.sh
```

11. Как определяются функции в языке программирования bash?

```
function name() { }
```

12. Каким образом можно выяснить, является файл каталогом или обычным файлом?

```
test -d
test -f
```

13. Каково назначение команд set, typeset и unset?

- set — установка параметров и массивов;
- typeset — объявление типа переменной;

- `unset` — удаление переменной или функции.
-

14. Как передаются параметры в командные файлы?

Через `$1`, `$2`, `$#`.

15. Назовите специальные переменные языка `bash` и их назначение

- `$0` — имя скрипта;
- `$1` ... `$9` — параметры;
- `$#` — количество аргументов;
- `$*` — все аргументы;
- `$?` — код завершения;
- `$$` — PID процесса; ““